

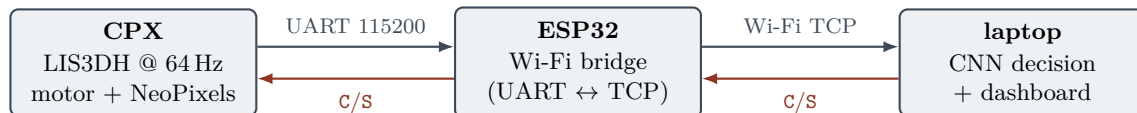
Pins & Connections

Parkinson's Closed-Loop Gait Garment — ENGF0031 Scenario 2

Circuit Playground Express + ESP32 (Wi-Fi bridge) + laptop (wireless)

The wearable is now **untethered**. The Circuit Playground Express (CPX) does all the sensing and drives the cue; a small **ESP32** relays data to and from the laptop over **Wi-Fi**; and the laptop runs the CNN that makes the freeze decision. CPX and ESP32 talk over a short **UART** link — both are 3.3 V logic, so they connect directly with *no level shifter*. The motor-driver wiring is unchanged from the earlier builds; only the *link to the laptop* has changed (a Wi-Fi hop via the ESP32 instead of a USB cable).

System overview — where the data goes



Accelerometer stream flows right (*ax, ay, az* at 64 Hz); the freeze-cue command (C/S) returns left.

Board pin map — Circuit Playground Express

Pin / pad	Direction	Connects to	Purpose
A1	digital out	1 kΩ → Q1 base	switches the vibration motor (the cue)
A7	UART TX (out)	ESP32 GPIO16 (RX2)	Serial1 TX — streams <i>ax, ay, az</i> to the ESP32
A6	UART RX (in)	ESP32 GPIO17 (TX2)	Serial1 RX — receives the C/S cue
VOUT	power (3.3 V)	motor (+), diode cathode	board-regulated rail; supplies the motor
GND	power (0 V)	Q1 emitter, motor return	common ground — shared with the ESP32
A0	—	left open (unused)	onboard speaker/DAC pin — avoid
micro-USB	5 V in	USB power bank / charger	power only now; optional debug serial
Button A	onboard input	—	hold still + press to calibrate the energy floor
Slide switch	onboard input	—	cue master-enable (wearer can silence it)
NeoPixels ×10	onboard output	—	state indicator (see legend below)
LIS3DH accel	onboard (I ² C)	—	3-axis accelerometer, sampled at 64 Hz

CPX ↔ ESP32 — the UART link

Three wires carry exactly the bytes the old USB cable used to. **TX** and **RX cross over**, and the two boards **must share a ground**. Both run at 3.3 V, so the lines connect directly — no level shifter, no divider.

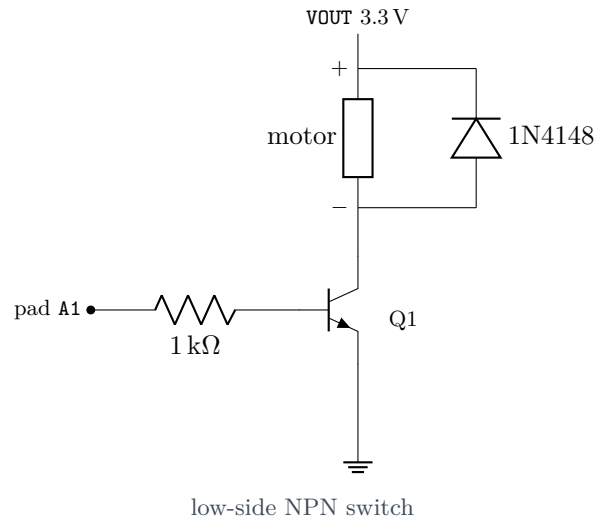
CPX pad	ESP32 pin	Signal
A7 (TX / D1) →	GPIO16 (RX2)	accel stream ax, ay, az , 64 Hz
A6 (RX / D0) ←	GPIO17 (TX2)	cue command C / S
GND —	GND	common ground — mandatory

Notes. 115200 baud, 8N1 (same framing as the old USB link). **GPIO16/17** are the ESP32's default **Serial2** pins on a WROOM DevKitC; on WROVER modules those pins are used by the PSRAM, so pick two free GPIOs (e.g. 25/26) and pass them to `Serial2.begin()`. Keep the wires short and do *not* also link the two 3.3 V rails — power each board on its own.

Motor driver — off-board components

A CPX pad can only source a few mA, but the coin motor draws ~ 90 mA, so pad **A1** *switches* a transistor that passes the motor current from **VOUT**:

- A1 → R1 (1 k Ω) → Q1 **base**
- Q1 **emitter** → GND
- Q1 **collector** → motor (–)
- motor (+) → VOUT (3.3 V)
- D1 across the motor, **cathode** → motor (+)/VOUT



Ref	Part	Value / type	Function
Q1	NPN BJT	2N2222 / BC547	low-side switch driven by A1
R1	resistor	1 k Ω	limits base current from the 3.3 V pad
D1	diode	1N4148	flyback — clamps the motor's inductive spike at switch-off
M1	coin vibration motor	~ 3 V, ~ 90 mA	the rhythmic vibrotactile gait cue

ESP32 connections

ESP32 pin	Direction	Connects to	Purpose
GPIO16 (RX2)	UART in	CPX A7 (TX)	receives the accel stream from the CPX
GPIO17 (TX2)	UART out	CPX A6 (RX)	sends the C/S cue to the CPX
GND	power (0 V)	CPX GND, battery (–)	common ground
5V / VIN	power in	USB power bank / battery (+)	board supply (on-board 3.3 V regulator)
Wi-Fi radio	—	laptop network	TCP socket to the laptop, ~ 64 Hz

Host link — Wi-Fi to the laptop

The ESP32 joins the same Wi-Fi network as the laptop (or the laptop's own hotspot) and opens a **TCP socket** to it. It forwards every accelerometer line it gets from the CPX and relays the laptop's cue commands back. The payloads are unchanged from the old USB link — only the transport differs,

so everything downstream on the laptop (the CNN, the dashboard) is untouched: the laptop simply reads a **socket** instead of `/dev/ttyACM0`.

Direction	Payload	Rate / format
CPX → ESP32 → laptop	<code>ax,ay,az</code> (newline-terminated)	64 Hz, three int16 in milli-g (CSV)
laptop → ESP32 → CPX	<code>C</code> = start cue, <code>S</code> = stop cue	one character, sent on a state change

In the *standalone* build the CPX computes the Freeze Index itself and drives the motor with no ESP32 or laptop attached — the wireless link is only for the CNN (full-PyTorch) build.

NeoPixel state legend

- **Green** — walking / monitoring
- **Red (pulsing)** — actively cueing
- **Dim teal** — standing still (gate holding the cue off)
- **Blue** — calibrating the still-floor

Power & safety. The wearable now runs on **battery** (a USB power bank or a LiPo) — power the CPX and the ESP32 each from their own 5 V input and **tie their grounds together**; do not cross-link the two 3.3 V rails. The motor’s ~90 mA is switched from `VOUT`; the **1N4148 flyback diode is mandatory** to protect Q1 from the inductive kick at switch-off. The laptop is self-powered (its own battery/charger) and shares no power rail with the wearable. `A0` is left unconnected (onboard speaker/DAC), and the UART link needs a **shared ground** and short wires to stay reliable.